

Sisteme de numerație | Operații pe biți

Andrei Mihai Ciobanu

Algolymp Summer School

Cuprins

Baze de numerație

Reprezentarea numerelor în calculator

Conversii între baze

Operații pe biți

Aplicații

Funcții utile

Recapitulare

Baze de numerație

Baze de numerație

Un număr poate fi scris în orice **bază** b, folosind cifre de la 0 la $b-1$:

Baza	Cifre	Exemplu	Sumă de puteri
10 (zecimal)	$0 \ldots 9$	425_{10}	$4 \cdot 10^2 + 2 \cdot 10 + 5$
2 (binar)	$0,\ 1$	$1101_2 = 13$	$2^3 + 2^2 + 2^0$
16 (hexa)	$0 \ldots 9,\ A \ldots F$	$\mathrm{FF}_{16} = 255$	$15 \cdot 16 + 15$

- În hexazecimal: $A = 10,\ B = 11,\ C = 12,\ D = 13,\ E = 14,\ F = 15$
- În C++: `0b1101` = binar, `0xFF` = hexazecimal

Întrebare

Care este valoarea lui $2\mathrm{A}_{16}$ în baza 10?

$2\mathrm{A}_{16} = 2 \cdot 16 + 10 = 42$

Reprezentarea numerelor în calculator

Reprezentarea numerelor pe biți

Calculatoarele lucrează doar cu **biți** — cifre care pot fi doar 0 sau 1.

Întrebare

Cum se poate reprezenta numărul 13 folosind doar biți?

Orice număr întreg se reprezintă ca o **sumă de puteri ale lui 2**:

$$13 = 2^3 + 2^2 + 2^0 = 8 + 4 + 1$$

Reprezentare pe 8 biți (exemplu: 13)

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	0	0	1	1	0	1

- `int` (32 biți) $\rightarrow$ valori de la -2^{31} la $2^{31} - 1 \approx 2 \cdot 10^9$
- `long long` (64 biți) $\rightarrow$ valori de la -2^{63} la $2^{63} - 1 \approx 9 \cdot 10^{18}$

Conversii între baze

Conversia din baza 10 în baza b

Întrebare

Cum se obține reprezentarea lui 13_{10} în baza 2?

Hint: Se împarte repetat numărul la 2 și se reține resturile.

- $13 \div 2 = 6$ rest 1
- $6 \div 2 = 3$ rest 0
- $3 \div 2 = 1$ rest 1
- $1 \div 2 = 0$ rest 1
- $\Rightarrow \ 13_{10} = 1101_2$

Algoritmul: Se împarte repetat la b, iar resturile se citesc **de jos în sus**.

Conversia din baza 10 în baza b — implementare

```cpp
// Scrie in digits cifrele lui n in baza b, de la cel mai
// semnificativ, si returneaza numarul de cifre
int toBase(int n, int b, int digits[]) {
    int cnt = 0;
    do {
        digits[cnt++] = n % b;
        n /= b;
    } while (n > 0);
    reverse(digits, digits + cnt); // inverseaza ordinea elementelor din intervalul [0, cnt)
    return cnt;
}
```

- Resturile se obțin de la cifra cea mai puțin semnificativă, de aceea cifrele se inversează
- Bucla `do ... while` asigură cifra 0 pentru $n = 0$
- Tabloul `digits` trebuie să aibă dimensiune suficientă

▷ *reverse — funcție din STL*

Conversia din baza b în baza 10

Întrebare

Cum se calculează valoarea lui 1101_2 dacă se prelucrează cifrele de la stânga la dreapta?

$$1101_2 = ((1 \cdot 2 + 1) \cdot 2 + 0) \cdot 2 + 1 = 13$$

Pas cu pas:

- $result = 1$
- $result = 1 \cdot 2 + 1 = 3$
- $result = 3 \cdot 2 + 0 = 6$
- $result = 6 \cdot 2 + 1 = 13$ ✓

```
1 // digits[0..cnt-1]: cifrele in baza b, de la cel mai semnificativ
2 int fromBase(int digits[], int cnt, int b) {
3     int result = 0;
4     for (int i = 0; i < cnt; i++) {
5         result = result * b + digits[i];
6     }
7     return result;
8 }
```

La fiecare pas, rezultatul parțial se înmulțește cu b și se adaugă următoarea cifră.

Operații pe biți

Biții unui număr se numerotează **de la dreapta la stânga**, începând cu 0.

Întrebare

Care sunt biții setați ai lui $x = 89 = 1011001_2$ și ce valoare are fiecare?

Indice bit	6	5	4	3	2	1	0
Bit	1	0	1	1	0	0	1
Valoare	64	32	16	8	4	2	1

Biții setați: 6, 4, 3, 0 $\Rightarrow$ $2^6 + 2^4 + 2^3 + 2^0 = 64 + 16 + 8 + 1 = 89$

- **LSB** (*Least Significant Bit*) — bitul cel mai puțin semnificativ: bitul 0, cel mai din dreapta
- **MSB** (*Most Significant Bit*) — bitul cel mai semnificativ: bitul cu cel mai mare indice setat

NOT — ~

Inversează **fiecare bit**: 0 devine 1, 1 devine 0.

p	0	1
$\sim p$	1	0

Întrebare

Care este $\sim a$ pentru $a = 10 = 1010_2$?

a:	1	0	1	0
$\sim a$:	0	1	0	1

```
1 int a = 0b1010; // 10
2 int r = ~a;    // -11
```

Pe 32 de biți: $\sim a = -(a+1)$, deci $\sim 10 = -11$.

▷ *Complement față de 2 — modul în care calculatoarele stochează numerele negative*

AND — &

Rezultat 1 dacă **ambii** biți corespunzători sunt 1.

p	q	$p \,\&\, q$
0	0	0
0	1	0
1	0	0
1	1	1

Întrebare

Care este rezultatul $a \,\&\, b$ pentru $a = 1010_2$, $b = 1100_2$?

a:	1	0	1	0
b:	1	1	0	0
$a \,\&\, b$:	1	0	0	0

```
1 int a = 0b1010, b = 0b1100;
2 int r = a & b; // 0b1000 = 8
```

OR — |

Rezultat 1 dacă **cel puțin unul** dintre biți este 1.

p	q	$p \mid q$
0	0	0
0	1	1
1	0	1
1	1	1

Întrebare

Care este rezultatul $a \mid b$ pentru $a = 1010_2$, $b = 1100_2$?

a:	1	0	1	0
b:	1	1	0	0
$a \mid b$:	1	1	1	0

```
int a = 0b1010, b = 0b1100;
int r = a | b; // 0b1110 = 14
```

XOR — ^

Rezultat 1 dacă biții sunt **diferiți**.

p	q	$p \oplus q$
0	0	0
0	1	1
1	0	1
1	1	0

Proprietăți XOR: $a \oplus a = 0$; $a \oplus 0 = a$; XOR este comutativ și asociativ.

Întrebare

Care este rezultatul $a \oplus b$ pentru $a = 1010_2$, $b = 1100_2$?

a:	1	0	1	0
b:	1	1	0	0
$a \oplus b$:	0	1	1	0

```
1 int a = 0b1010, b = 0b1100;
2 int r = a ^ b; // 0b0110 = 6
```

Deplasări pe biți — left shift și right shift

Întrebare

Ce efect au deplasările la stânga și la dreapta asupra valorii unui număr?

- `x << k` $= x \cdot 2^k$ (deplasare la stânga)
- `x >> k` $= \lfloor x/2^k \rfloor$ (deplasare la dreapta)

```
1 int x = 1;     // 0b00000001
2 int a = x << 3; // 0b00001000 = 8 (x * 2^3)
3 int b = x >> 0; // 0b00000001 = 1
4
5 int y = 12;    // 0b00001100
6 int c = y << 1; // 0b00011000 = 24 (y * 2)
7 int d = y >> 1; // 0b00000110 = 6 (y / 2)
8 int e = y >> 3; // 0b00000001 = 1 (y / 8)
```

Atenție: `1 << k` dă overflow pentru $k \geq 31$ pe `int`; pentru `long long` se folosește `1LL << k`.

Aplicații

Verificarea unui bit

Întrebare

Cum se verifică dacă al k-lea bit al lui x este setat?

Hint: Se aduce bitul k pe poziția 0 prin deplasare, apoi se izolează cu o mască.

```
// Verificam daca al k-lea bit este setat
bool isSet(int x, int k) {
    return (x >> k) & 1;
}

// Caz special: x impar <=> bitul 0 setat
bool isOdd(int x) {
    return x & 1;
}
```

- Exemplu: $x = 13 = 1101_2$, $k = 2$: $x \gg 2 = 0011_2$, iar $0011_2 \,\&\, 1 = 1 \Rightarrow$ bitul 2 este setat
- Pentru $k = 1$: $x \gg 1 = 0110_2$, iar $0110_2 \,\&\, 1 = 0 \Rightarrow$ bitul 1 nu este setat

Setarea, resetarea și inversarea unui bit

Întrebare

Cum se setează, se resetează și se inversează al k-lea bit al lui x?

- Masca `1 << k` are setat doar bitul k
- `|` setează, `& ~` resetează, `^` inversează bitul selectat de mască

```
1  // Setam al k-lea bit (il facem 1)
2  int setBit(int x, int k) {
3      return x | (1 << k);
4  }
5
6  // Resetam al k-lea bit (il facem 0)
7  int clearBit(int x, int k) {
8      return x & ~(1 << k);
9  }
10
11 // Flippuim al k-lea bit (0->1, 1->0)
12 int flipBit(int x, int k) {
13     return x ^ (1 << k);
14 }
```

Ultimii k biți și puterile lui 2

```cpp
// Ultimii k biti ai lui x (ex: k=3 => masca 0b0111)
int lastKBits(int x, int k) {
    return x & ((1 << k) - 1);
}
```

Întrebare

Cum se verifică dacă n este putere a lui 2?

Hint: Ce efect are $n \,\&\, (n-1)$ asupra celui mai din dreapta bit de 1?

- O putere a lui 2 are exact un singur bit de 1
- $n \,\&\, (n-1)$ elimină cel mai din dreapta bit de 1
- Exemplu: $4 = 100_2$, $3 = 011_2$, $4 \,\&\, 3 = 0$ ✓

```cpp
bool isPow2(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

Funcții utile

Popcount — numărul de biți de 1

Întrebare

Cum se determină numărul de biți de 1 ai unui număr?

```
// Implementare manuala: O(nr. biti)
int popcount(int x) {
    int cnt = 0;
    while (x > 0) { cnt += x & 1; x >>= 1; }
    return cnt;
}
```

```
// Built-in GCC (O(1), foarte rapid)
int cnt = __builtin_popcount(x); // pentru int
int cnt = __builtin_popcountll(x); // pentru long long
```

- Exemplu: `popcount(13)` = `popcount(0b1101)` = 3
- Built-in-urile `__builtin_*` sunt specifice GCC — funcționează pe majoritatea compilatoarelor de concurs (g++), dar nu sunt garantate pe MSVC

Leading zeros și trailing zeros

- **Leading zeros**: biți de 0 **de la stânga** până la primul bit de 1
- **Trailing zeros**: biți de 0 **de la dreapta** până la primul bit de 1

Întrebare

Câți biți de 0 sunt la stânga, respectiv la dreapta primului bit de 1 pentru $x = 12 = 00001100_2$ (pe 32 de biți)?

Leading zeros $= 28$ (primii 28 de biți sunt 0), trailing zeros $= 2$.

```
int lz = __builtin_clz(x); // leading zeros (int)
int tz = __builtin_ctz(x); // trailing zeros (int)
// Variante pentru long long: __builtin_clzll, __builtin_ctzll

// Atentie: __builtin_clz(0) si __builtin_ctz(0) sunt nedefinite!
```

- Trailing zeros al lui x = indicele celui mai mic bit setat al lui x
- Exemplu: `__builtin_ctz(12)` = `__builtin_ctz(0b1100)` $= 2$

Lowest set bit — `x & (-x)`

Întrebare

Cum se izolează cel mai din dreapta bit de 1 (**lowest set bit**) al lui x?

Hint: Reprezentarea **complement față de 2**: $-x = \sim x + 1$ (se inversează toți biții și se adaugă 1).

```
int lowestBit(int x) { return x & (-x); }
// x = 12 = 0b1100 => lowestBit(12) = 0b0100 = 4
```

- La adunarea lui 1, carry-ul se propagă exact până la primul bit de 1 din x
- Astfel $-x$ coincide cu x începând de la primul bit de 1 (inclusiv), iar restul este inversat
- $x \,\&\, (-x)$ păstrează doar acel bit comun

Exemplu: $x = 12 = 1100_2$, $-x = 0100_2$ (în 4 biți), $x \,\&\, (-x) = 0100_2 = 4$

Recapitulare

Recapitulare — Operații uzuale

Operație	Cod C++
Bitul k e setat?	`(x >> k)& 1`
x e impar?	`x & 1`
Setează bitul k	`x | (1 << k)`
Resetează bitul k	`x & ~(1 << k)`
Flip bitul k	`x ^ (1 << k)`
Ultimii k biți	`x & ((1 << k)- 1)`
n e putere de 2?	`n > 0 && !(n & (n-1))`
Lowest set bit	`x & (-x)`
Număr de biți de 1	`__builtin_popcount(x)`
Trailing zeros	`__builtin_ctz(x)`

Probleme de antrenament

Ușoare

1. **getbit** (pbinfo.ro)
2. **setbit0** (pbinfo.ro)
3. **setbit1** (pbinfo.ro)
4. **puterebiti** (pbinfo.ro)

Medii

1. **Base Bit Report** (AlgoCode #4271)
2. **addone** (pbinfo.ro)
3. **invbits** (pbinfo.ro)

Resurse

1. **pbinfo.ro** — Baze de numerație (RO)
2. **pbinfo.ro** — Operații pe biți (RO, ușor de înțeles)
3. **cp-algorithms.com** — Bit manipulation (EN)
4. **cppi.sync.ro** — Material mai stufos de Marius Nicoli (RO)
5. **wikipedia.org** — Two's complement (EN)